LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN

Advanced Topics in Parallel Computing

Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

Memory Consistency Models

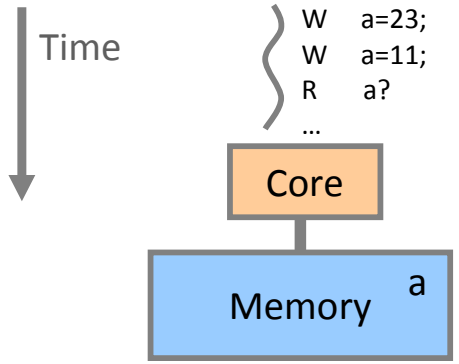


Outline for Today's Lecture

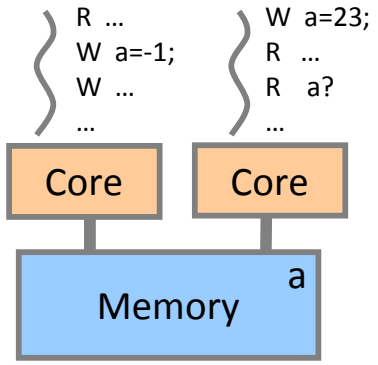
- Memory consistency models
  - Review of **Sequential Consistency**
  - What should a memory consistency model offer?
  - Motivation for relaxed consistency
  - Synchronizing operations
- Example consistency models
  - SC – Sequential Consistency
  - TSO – Total Store Order
  - Relaxed/Weak Consistency
- Cache Coherence vs. Memory Consistency
  - Coherence invariants
  - Connection between coherence and consistency

Motivation

- What should we expect from a memory system?
  - For a **sequential** computer: A **read operation** of a memory location should return the value of the **most recent write to that location**



  - For a **parallel** computer: ……?

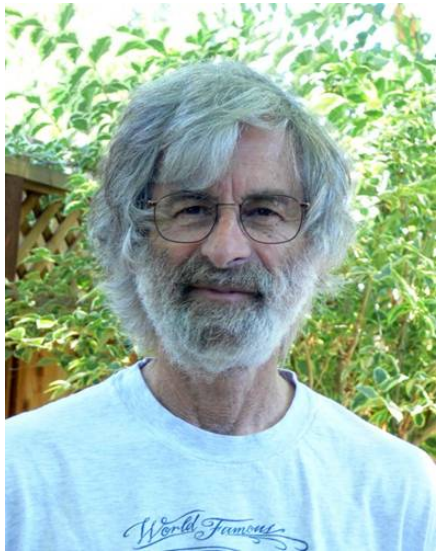


Sequential Consistency (1)

- One natural and intuitive model: A read operation of a memory location **by some thread** should return the value of the **most recent write** operation (**by any thread**) to that location
  - Implication: read/write operations should be **atomic**
  - Implication: there should be a **total order** of all memory accesses

- This intuitive model is also known as **sequential consistency** and was formalized by Leslie Lamport (1979)

Leslie Lamport. *How to make a multiprocessor computer that correctly executes multiprocess programs*. IEEE Transactions on Computers, C-28(9):690–691, September 1979.



Sequential Consistency (2)

- **Definition**: A multiprocessor [multicore processor] is sequentially consistent, if…
  - "… the result of any execution is the same as if the operations of all processors [cores] were executed in **some sequential order**, and the operations of each individual processor [core] appear in this sequence in the **order specified by its program**."
  - The *order specified by its program* is also called **Program Order (PO)**

Sequential Consistency (3)

- Operational model / visualization of SC

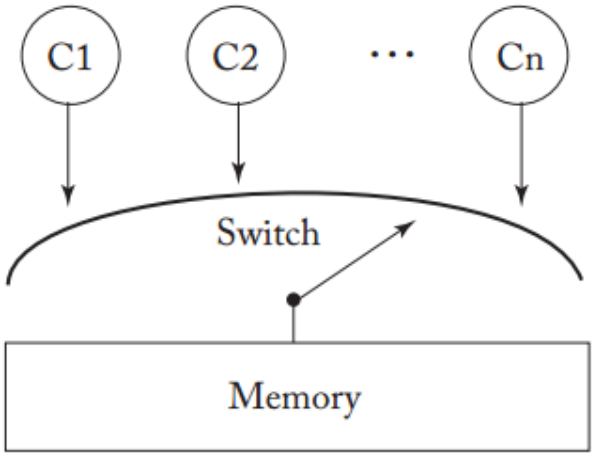


Image Source: V. Nagarajan, D. Sorin, M. Hill, and D. Wood, "A Primer on Memory Consistency and Cache Coherence Second Edition"

- Consequences of SC:
  - All memory operations are atomic (performed to completion and in isolation of the others)
  - A read always returns the value of the most recent write
  - Program Order (PO) in each thread is respected
  - The total order of all memory accesses is some interleaving of the operations of each individual processor

Example and Visualization of SC

```
// initial conditions
val=0; flag=0; y=0;

// Thread 0:

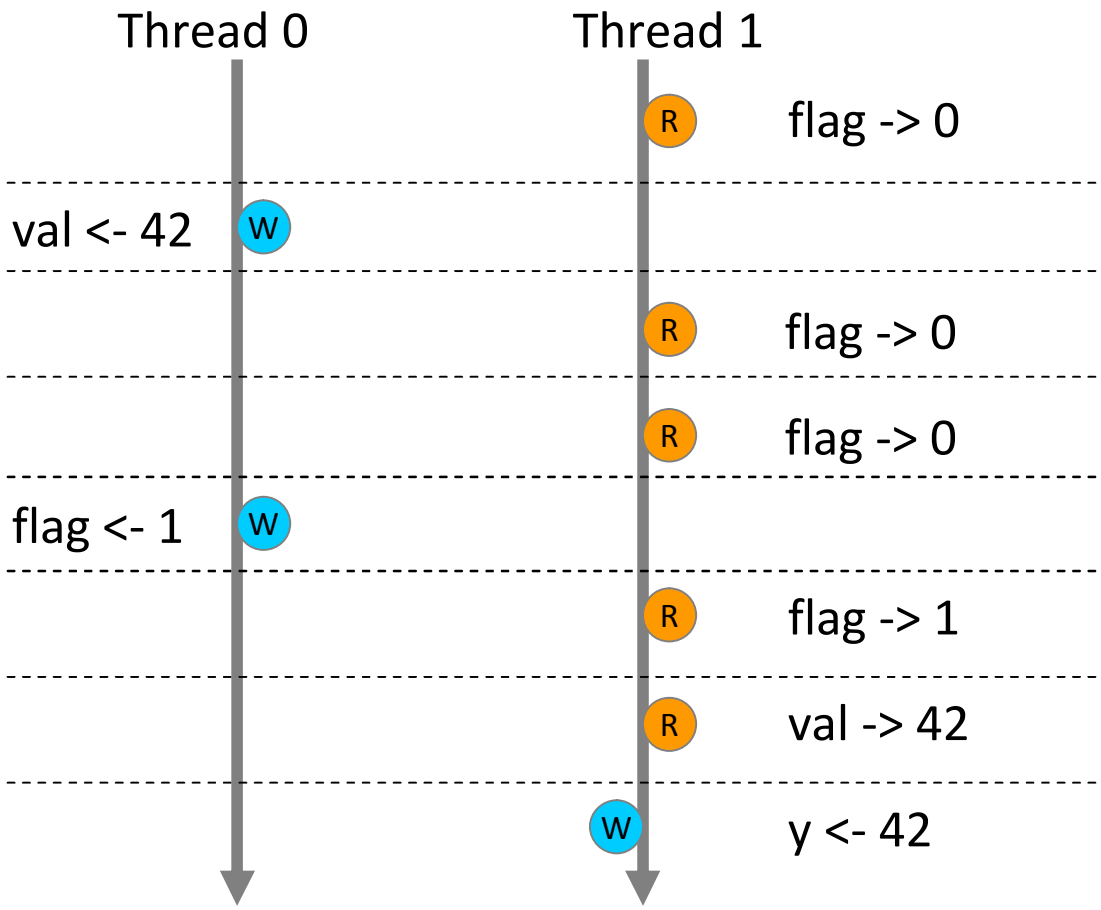
val = 42;
flag = 1;
```

```
// Thread 1:

while(!flag) {} // spin

y = val; // read val
```

A possible execution of this program on a SC system (an SC execution):



Discussion of Previous Example

- The scenario on the previous slide is an example for **shared memory synchronization** between threads (or processes), called **condition synchronization**
  - Wait for a condition to be met, where "condition" is usually expressed by referring to the **values of variables** in shared memory
  - **Example**: perform some sort of initialization, reading from disk, etc. before setting a flag to indicate the end of the initialization phase
  - **Example:** synchronization barriers, separating computational phases
- What could go wrong? (Is SC already the end of the story?)
- The example shows that we need a well-defined model of how memory should work
  - SC is one such model
  - SC is well-defined and relatively easy to understand
  - SC is, however, not the only memory consistency model

Definitions (1)

- We're concerned with **shared memory** systems, where each of the processor cores may read from and write to a **single shared address space**

- A **memory consistency model** conceptually specifies how loads and stores act upon shared memory; it defines the (architecturally visible) **correct shared memory behavior**
  - For a program with specific input data, it **defines what values load operations may return and what possible final states of memory are allowed**
  - It tells programmers what behavior they can rely on, it tells HW implementers what they need to provide, it specifies limits to optimizations

- Other frequently used terms:
  - Memory model
  - Consistency model
  - Memory consistency

Definitions (2)

- **Related** but different concept: **cache coherence** – ensure that multiple copies of variables are kept up-to-date
  - Why multiple copies? Because systems use caches to speed up access latency (memory is slow)

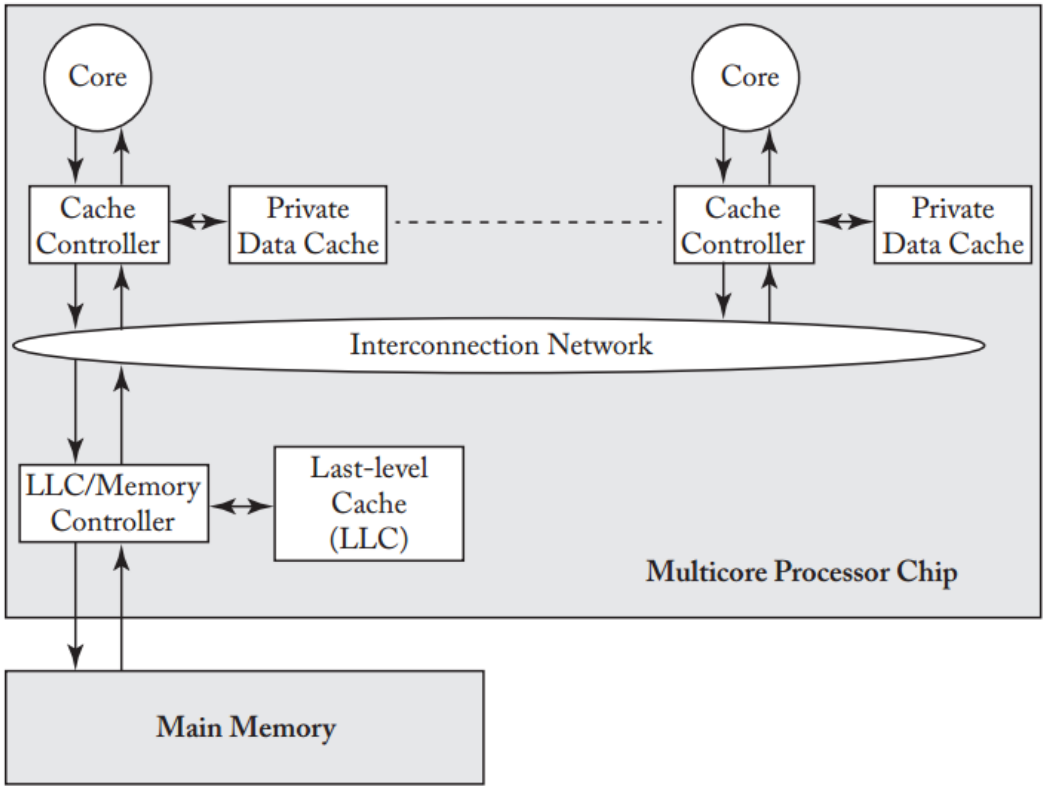
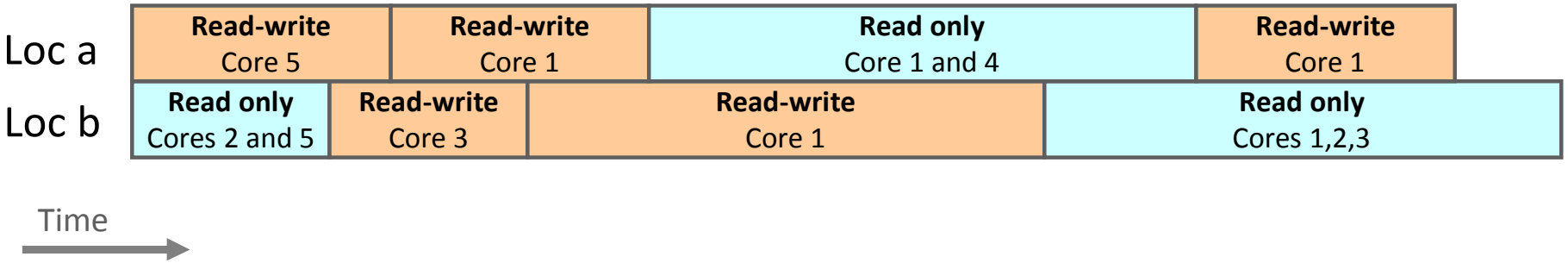


Image Source: V. Nagarajan, D. Sorin, M. Hill, and D. Wood, "A Primer on Memory Consistency and Cache Coherence Second Edition"

Coherence (1)

- One way to think about Cache Coherence is in terms of **access epochs**:
  - The lifetime of **each memory location** is divided into epochs
  - The coherence invariants (below) are maintained for each memory location separately

- Coherence Invariants
  - **Single-Writer Multiple-Reader** Invariant (SWMR): For any given memory location, at any given time, there is either one core that may **write (and read)** to the location or any number of cores that may **only read** the location
  - **Data-Value Invariant**: the value of the memory location at the start of an epoch is the same as the value at the end of the last single-writer epoch



Coherence (2)

- Cache coherence protocols are used to enforce the coherence invariants
  - Protocols are implemented by cache controllers
  - Many variants have been developed over the years
  - Often based on state machines with states such as **M**odified, **O**wned, **E**xclusive, **S**hared, **I**nvalid
  - In reality, the coherence invariants are not maintained for each memory location individually, but on a cache block (line) basis (typically 64 Bytes)

- An operational model is **token coherence**
  - For a system with N cores, there are at least N tokens for **each memory location**
  - A core that wants to write to the memory location has to have **all** tokens
  - A core that has **at least 1** token may read the memory location

Coherence vs. Consistency

- Coherence alone doesn't give us enough guarantees to work with shared memory
  - Coherence is specified in terms of individual memory locations but **doesn't guarantee anything in terms of accesses to multiple memory locations**
  - Coherence guarantees that all accesses to a **particular memory location** appear to **occur in the same order**, from the perspective of **all threads**

```
// initial conditions
val=0; flag=0; y=0;

// Thread 0:

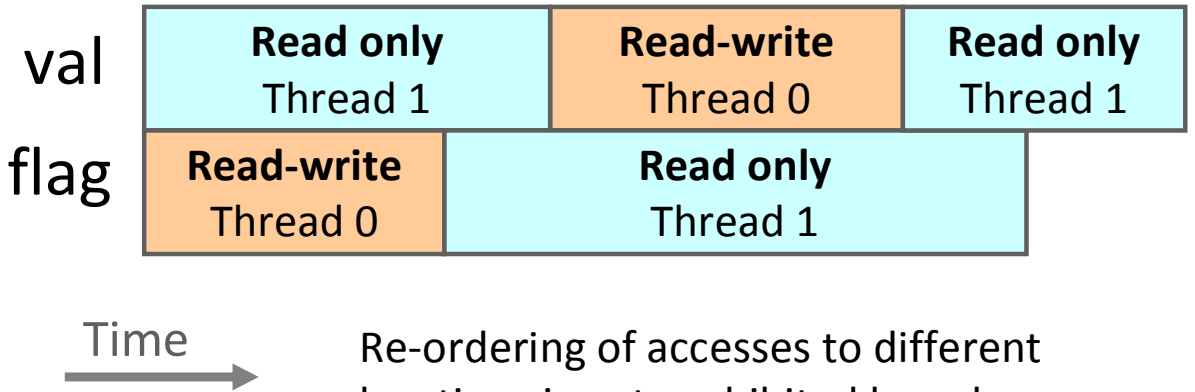
val = 42;
flag = 1;
```

```
// Thread 1:

while(!flag) {} // spin

y = val; // read val
```

In terms of **coherence** this would be a correct execution, but the ordering between accesses to **val** and **flag** is not respected



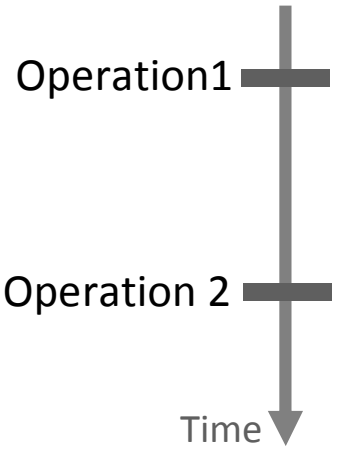
Re-ordering of accesses to different locations is not prohibited by coherence

The Story so far….

- Shared memory systems are composed of multiple cores
  - Cache coherence takes care of keeping cached copies of values up to date, automatically
  - But we need stronger guarantees about the behavior of shared memory to write useful (deterministic) programs
  - These guarantees are specified by the **memory consistency model**
- SC is the most "natural" and simple consistency model
  - Provides strong ordering guarantees
  - Other models have been developed, providing fewer guarantees
- Basic operations on shared memory
  - Reading and writing memory (loads and stores)
  - Updating: read-modify-write (RMW) operations, e.g., fetch-and-add – perform atomic updates on memory – useful for sync. algorithms, more later
  - Fences - limit reorderings, more later

Ordering Guarantees in SC (1)

- As we've seen, SC provides a total global order of all memory operations and the order of each thread's memory operation is respected in this total global order
  - We can summarize these ordering guarantees in the following table:



Operation 1 \ Operation 2	Load	Store	RMW
Load	X	X	X
Store	X	X	X
RMW	X	X	X

Image source: Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, David A. Wood *"A Primer on Memory Consistency and Cache Coherence, Second Edition"*

X: The ordering is respected, with respect to all threads

RMW: Read-modify-write operations, e.g., fetch-and-add, more on that later

Ordering Guarantees in SC (2)

```
// initial conditions
val=0; flag=0; y=0;

// Thread 0:

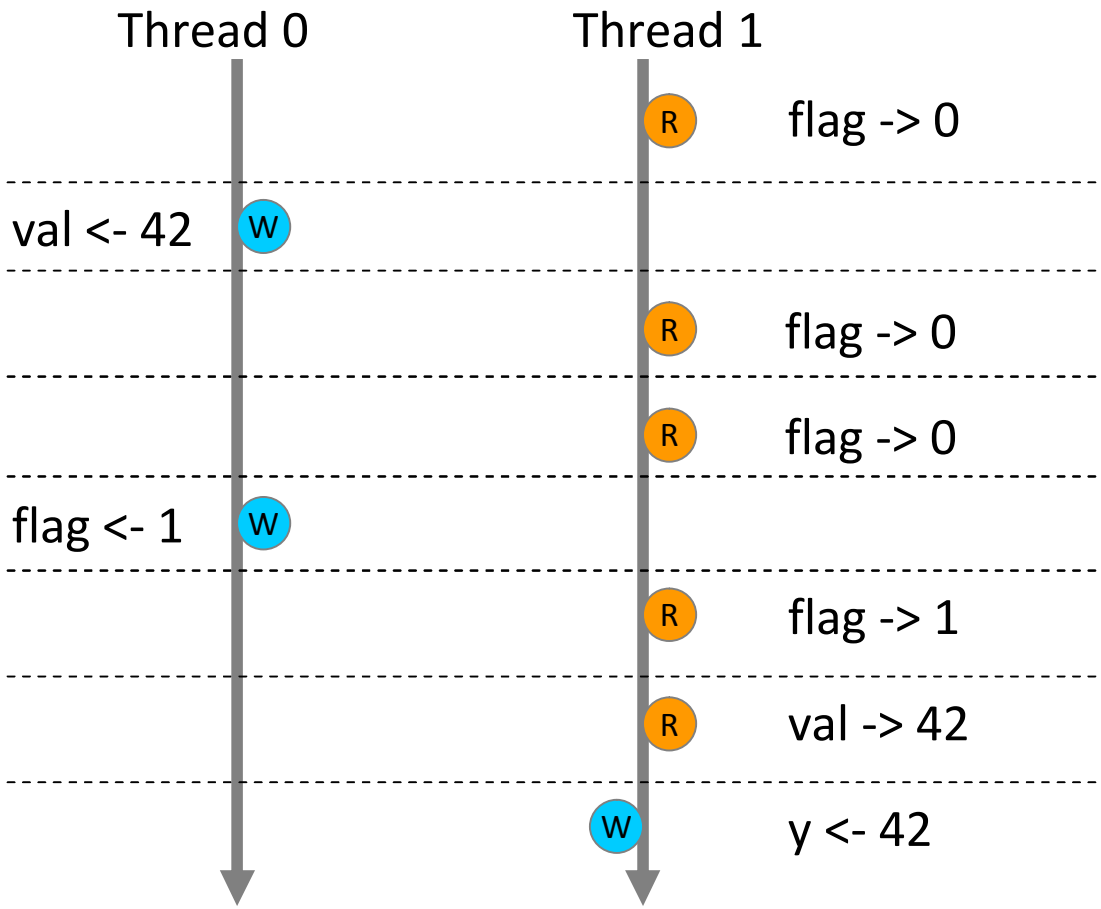
val = 42;
flag = 1;
```

```
// Thread 1:

while(!flag) {} // spin

y = val; // read val
```

A possible execution of this program on a SC system (an SC execution):



Note: A visualization in this form only makes sense for sequential consistency
Why? This picture implies a total global ordering of all memory accesses, and all threads see the same ordering.

Problems with Sequential Consistency

- In reality, most platforms **do not** support SC for basic memory operations
  - Why? Because it would prohibit many optimizations essential for performance on modern systems
- Examples:
  - **Out-of-order** (OoO) execution (CPU may reorder independent operations)
  - **Store buffers** (memory system too slow to handle many writes at once)
  - **Memory system** may have internal buffering
  - **Compilers** are permitted to reorder operations if effects are proven unobservable locally
- SC may also be overly restrictive for many idioms
  - Example: initialize many values, then set flag
  - In this case the order of initialization is not important, only that the flag is set last is essential

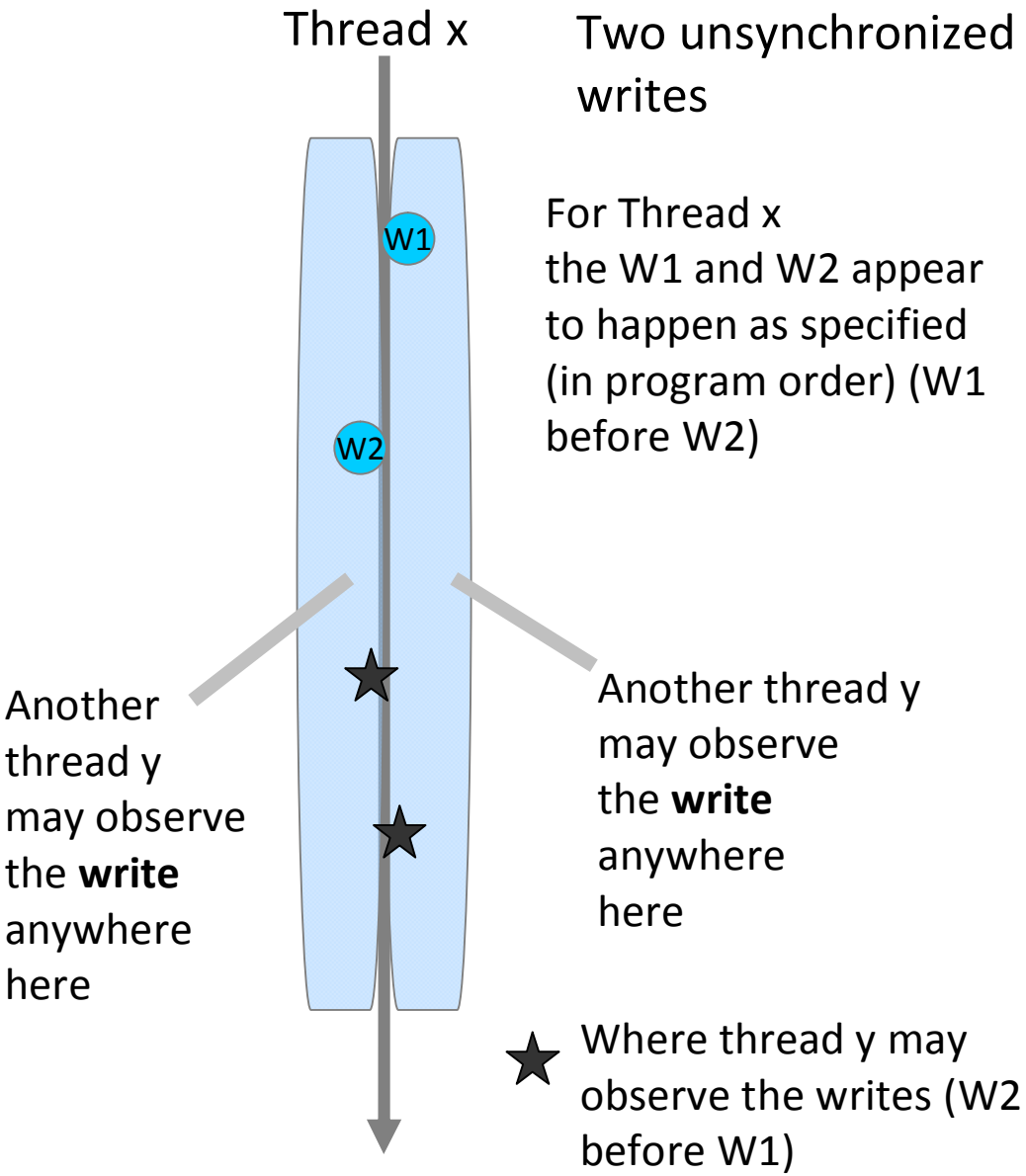
Alternatives to SC

- Result: **relaxed / weak memory models**
  - Within a thread, things stay the same: reads and writes **appear** to be executed as specified in the program (program order, PO)
  - I.e., within a thread, a read will **always** return the value of the most recent write
  - A different thread, however, **may observe different execution order**
  - For some relaxed memory models, all other threads observe the same execution order (a total global order of memory operations exists)
  - For other relaxed memory models, different threads may observe different execution orders
  - Need ways to **enforce certain ordering guarantees**, so called **synchronizing operations**

- First, let's look into how we can visualize the lack of ordering guarantees!

Visualizing Relaxed Ordering

- Visualization for a relaxed memory model
  - Locally (for each thread) R/W operations **appear** to be executed in the order in which they were specified (PO)
  - Other threads **may observe different orderings**
  - I.e., there is no observable "objective reality" for memory accesses
- Note:
  - This is just a visualization
  - Not necessarily a good basis for a formal description



Example:
Two unsynchronized writes

For Thread x
the W1 and W2 appear to happen as specified (in program order) (W1 before W2)

Previous Example under Relaxed Consistency

```
// initial conditions
val=0; flag=0; y=0;

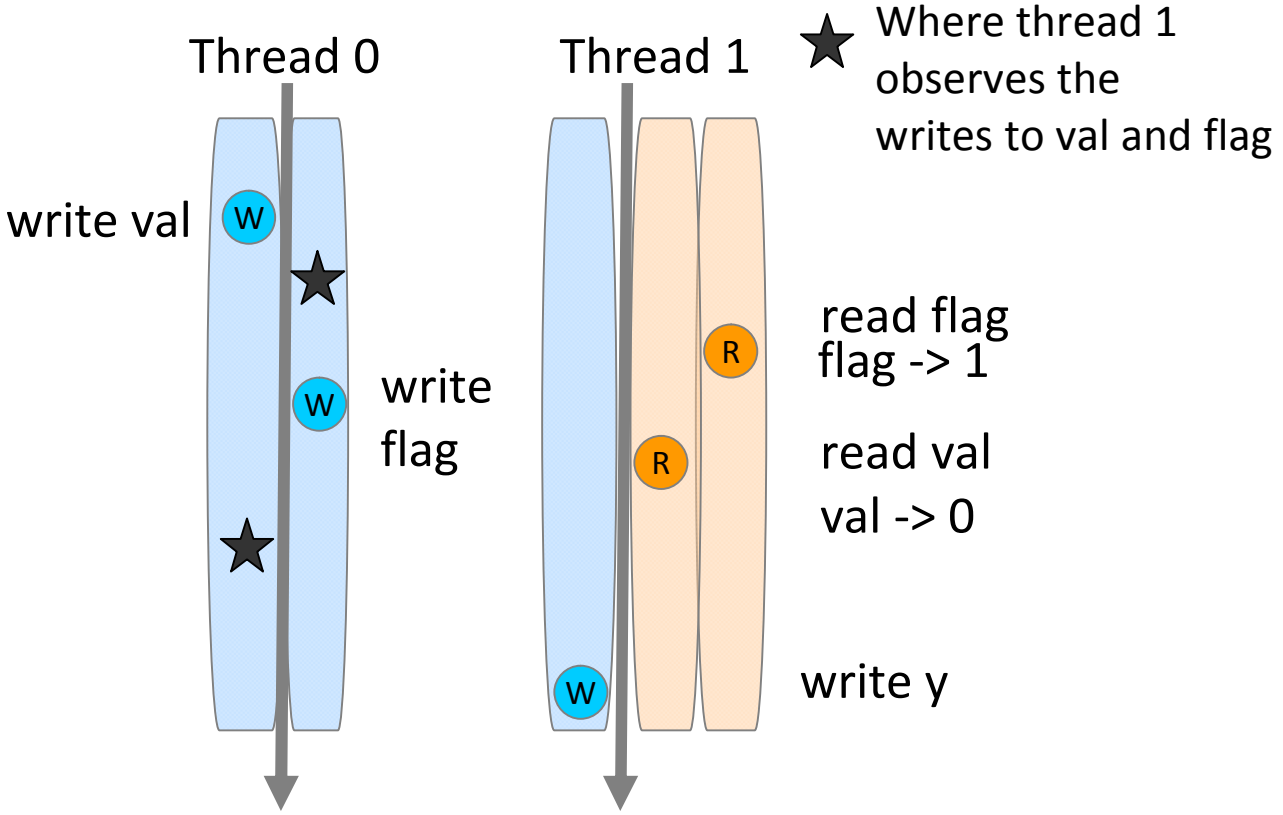
// Thread 0:

val = 42;
flag = 1;
```

```
// Thread 1:

while(!flag) {} // spin

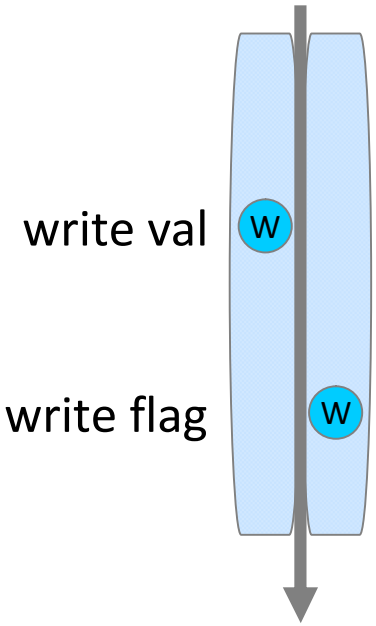
y = val;
```



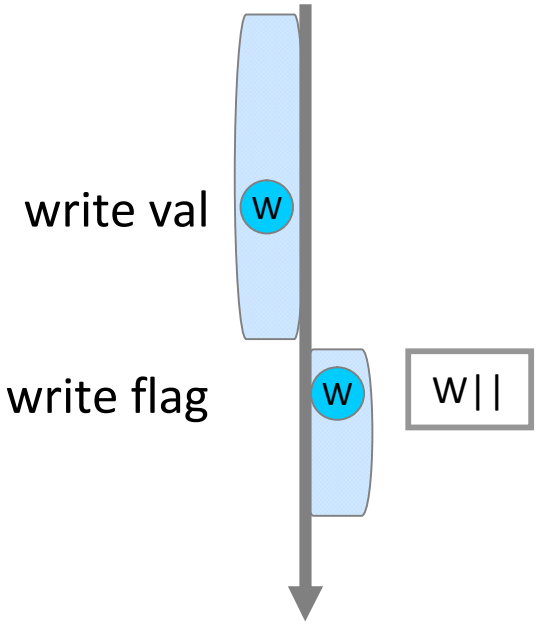
- Discussion:
  - We tried to establish an **invariant** in Thread 0 (first write val, then write flag) that we relied on in Thread 1
  - This doesn't work in relaxed memory models because different threads may observe different orderings
  - Consequence: We need ways to **limit reorderings**

Limiting Reordering for Relaxed Consistency

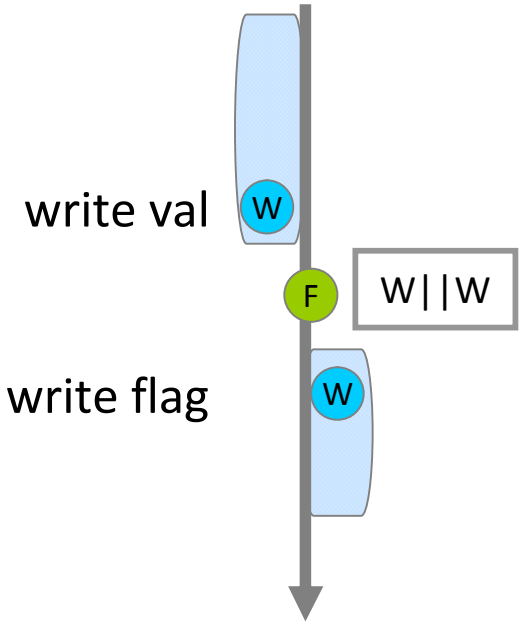
- We need to make sure that the write to val appears to happen before the write to flag, **from the point of view of all threads**



Using regular write operations, no ordering restrictions. Other threads may observe operations in any order



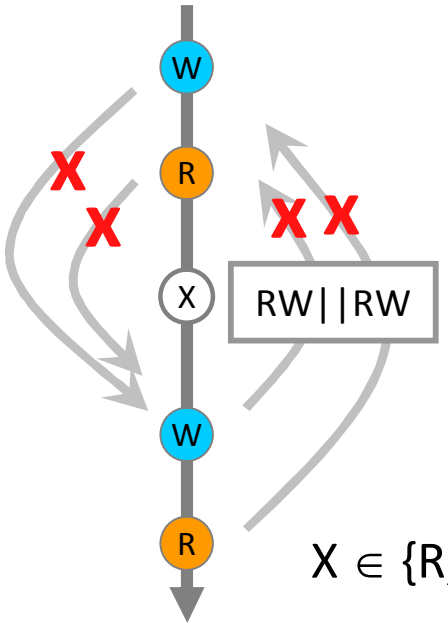
Ordering is guaranteed using a **synchronizing write** operation (to flag) "w||" specifies what kind of reorderings are **disallowed**



Ordering is guaranteed using a **synchronization operation** (**fence F**) "w||w" specifies what kind of reorderings are **disallowed**

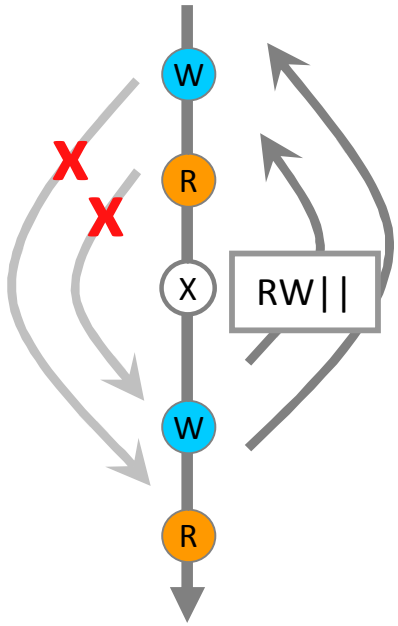
Limiting Reordering - Notation

- Notation for synchronizing instructions: P||S
  - P and S $\subseteq$ {R,W}; **P**: preceding, **S**: subsequent
  - Applies to a memory access operation (read, write) or standalone synchronization instruction (fence) and describes what kind of reorderings are **forbidden**

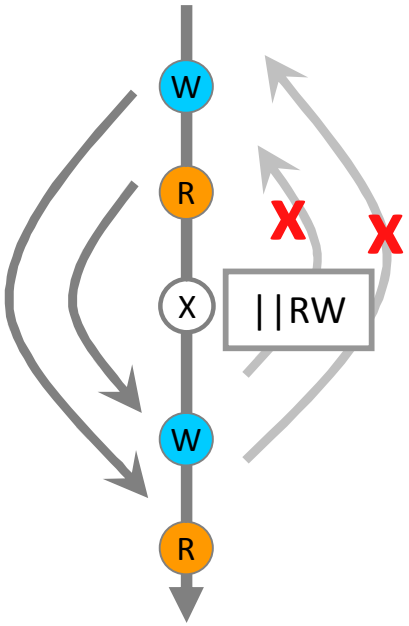


X ∈ {R, W, fence}

All reorderings around X are forbidden



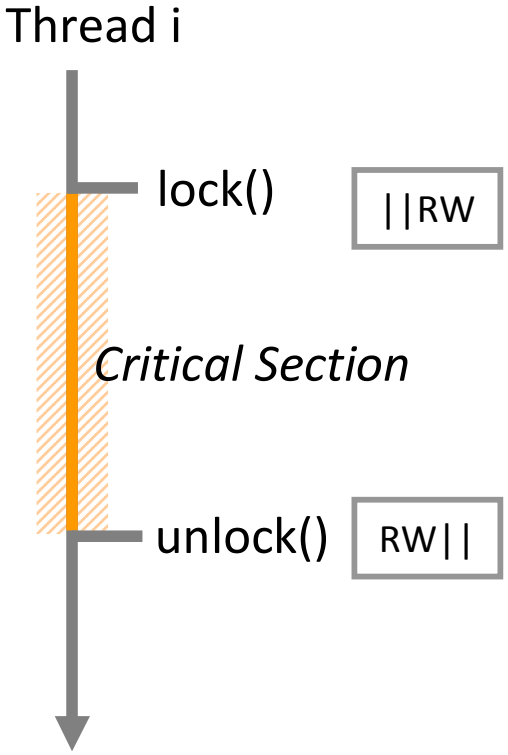
Preceding reads and writes can not be reordered to after the sync. operation X



Subsequent reads and writes can not be reordered to before to the sync. operation X

Two Important Orderings: Acquire and Release

- Critical sections are typically implemented using **lock** and **unlock** operations to protect shared resources (e.g., shared memory)
  - "acquire" lock on entry to critical section
  - "release" lock on exit from critical section



Acquire Ordering: "||RW" load or **"R||RW" fence**
Required because we need to prevent R,W operations to migrate **out of** the critical section (to **"before" the critical section**)

Release Ordering: "RW||" store or "RW||W" fence
Required because we need to prevent R,W operations to migrate **out of** the critical section (to **"after" the critical section**)

Synchronizing Instructions - Summary

- Synchronizing memory accesses establish local and global ordering:

- Local ordering:
  - They establish ordering relationships according to the P||S notation between ordinary/regular and sync. accesses, **from the point of view of all threads**

- Global ordering:
  - The sync. accesses appear in some **consistent total global order**, **from the point of view of all threads**

Memory Consistency in Hardware

- Hardware implementations differ in how relaxed the memory model is and what kind of sync. instructions are available

- SC supported in hardware
  - MIPS R10000, HP PA-RISC

- "Total Store Ordering" (TSO)
  - SPARC, Intel x86 (32 and 64 bit), IBM zSeries
  - Reads can overtake writes (W||R annotation is need to prevent this)
  - But all other orderings are respected (R||R , R||W, W||W is implied)

- "More relaxed" models
  - ARM, IBM Power, Intel IA-64 (Itanium)
  - All reorderings possible
  - ARM v8 and Itanium provide synchronizing load and store operations

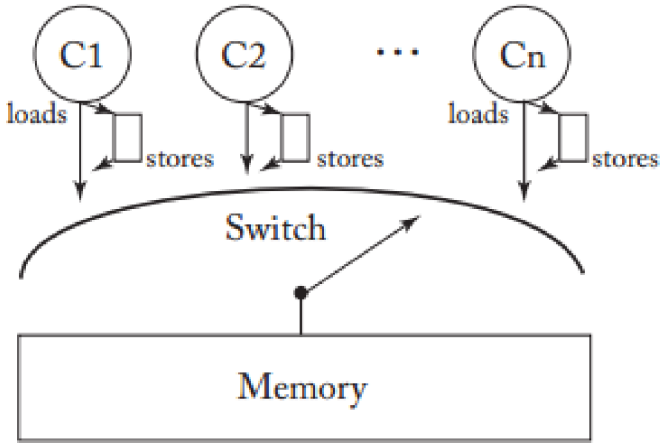
TSO (1)

- Total Store Order
  - TSO is almost as strict as SC
  - Like in SC, there is a **total global order** of all memory operations
  - All ordering guarantees are the same as in SC, except for one: **reads may overtake writes**
  - I.e., a write issued before a read in one thread may appear after the read in the total global order

- Motivation for TSO
  - Close to SC means fewer surprises for programmers
  - Memory system is slow compared to core
  - Introduce **store buffer** to avoid stalling the core just to write to memory
  - Writes go to store buffer, which eventually drains to memory (**FIFO** order)
  - Reads **check local store buffer** before consulting memory (so local PO is always respected), this is sometimes called **bypassing**, i.e., the value of the read is bypassed from the store buffer

TSO (2)

- Operational model for TSO
  - Just like SC, but with a per-core store buffer added



- Ordering guarantees for TSO

Operation 1 \ Operation 2	Load	Store	RMW	FENCE
Load	X	X	X	X
Store	B	X	X	X
RMW	X	X	X	X
FENCE	X	X	X	X

X: The ordering is respected
B: Bypassing for local loads
FENCE: Synchronizing operations

Image source: Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, David A. Wood *"A Primer on Memory Consistency and Cache Coherence, Second Edition"*

Relaxed / Weak Memory Models

- There are several different types of relaxed / weak memory models
  - For some of them, no total global memory can be defined, so a table of ordering guarantees (as for SC and TSO) doesn't make sense
  - All models provide ways to limit re-ordering though (fence operations and/or synchronizing read/writes)

- Takeaway
  - We need to program for the "worst case", i.e., most relaxed memory model
  - To do that we need to provide ordering restrictions, e.g., RW||
  - Compiler, runtime system, platform can turn those into platform-specific operations (fence, etc.) or can ignore them (if HW is strict enough)
  - We will later see how this is done in C++ and it's memory model

Using Ordering Restrictions

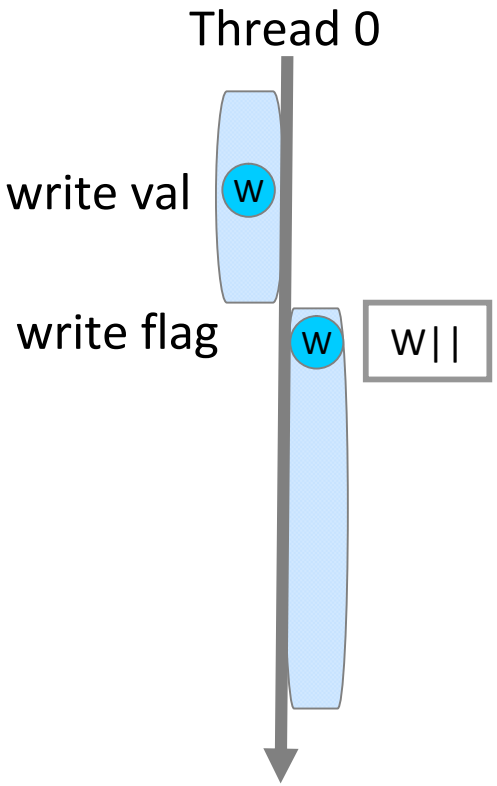
```
// initial conditions
val=0; flag=0; y=0;

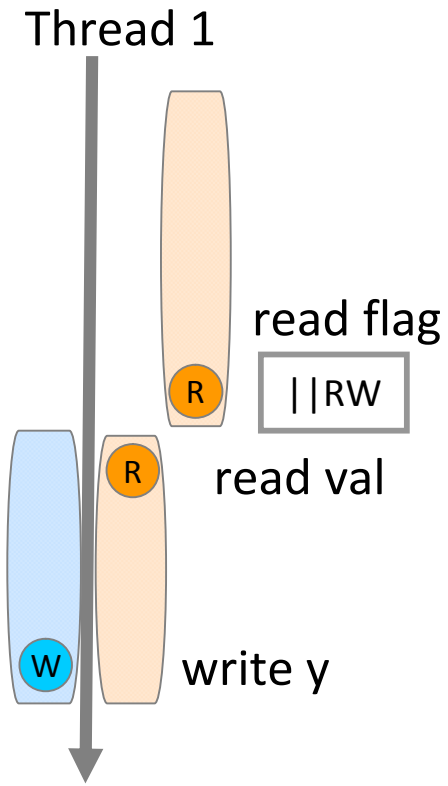
// Thread 0:
val.store(42); // ordinary store
flag.store(1, W||); // sync. store
```

```
// Thread 1:

while(!flag.load(||RW)) // sync. load
  { /* spin */ }

// ordinary load and store
y = val.load();
```





Another Example

```
// initial conditions
x=0; y=0;

// Thread 0:
x=1;
print(y);
```

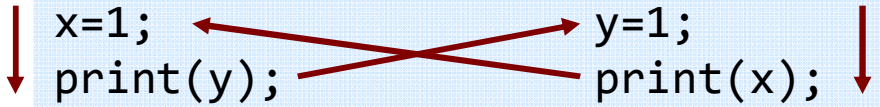
```
// Thread 1:
y=1;
print(x);
```

- Q: Can the output be x==0,y==0?
  - A: Without SC, yes.

- But as a parallel program, it contains an ordering loop:

```
// initial conditions
x=0; y=0;

// Thread 0:          // Thread 1:
x=1;                  y=1;
print(y);             print(x);
```

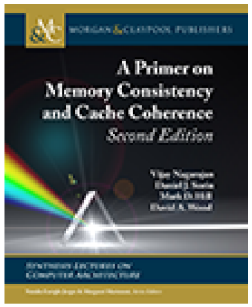


Up Next…

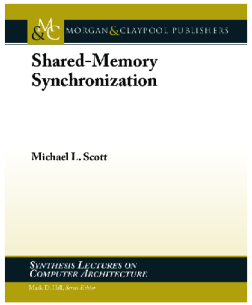
- How to work with the memory system from a high-level language
  - C++ atomic types and operations

- Shared memory synchronization
  - Lock algorithms and their implementation
  - Barriers and their implementation

Further Reading



- Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, David A. Wood *"A Primer on Memory Consistency and Cache Coherence, Second Edition"* Synthesis Lectures on Computer Architecture (2020)
  - Chapters 1-5



- Michael L. Scott *"Shared-memory synchronization"* Synthesis Lectures on Computer Architecture (2013)
  - Chapters 1-2